HITO 1 · PROYECTO FINAL INTEGRADOR

Gemelo Digital del ABB IRB 120

De la tabla Denavit-Hartenberg al modelo URDF validado en RViz con error vectorial $\Delta E < 10^{-6}$ m

GRUPO VEVAS · 4 integrantes

Robótica IMT-342 · Docente: M.Sc. Ing. Bernardo Quiroga Turdera · Gestión II-2026

OBJETIVO

Qué exige exactamente el Hito 1

El EC1 evalúa describir matemáticamente la pose y morfología de un robot de 6 GDL. El entregable no es “un robot que se ve bonito en RViz”: es una equivalencia demostrable entre tres representaciones.

Ejes D-H

Asignar los 6 sistemas locales e identificar los offsets de junta

URDF modular

XML sin errores de sintaxis, acoplando mallas .stl

$\Delta E < 10^{-6}$ m

Matriz de pruebas: TF de ROS vs. función analítica

Git secuencial

Historial de commits lógicos, sin volcado final

Pasaporte del hito: la pose [x, y, z] leída de RViz debe coincidir con la analítica en Python con $\Delta E < 10^{-6}$ m.

ARQUITECTURA

Físico → Matemático → Computacional

Cada etapa es una traducción con pérdida controlada de información. Entender qué se pierde en cada flecha es lo que evita los errores clásicos del laboratorio.

1

ROBOT FÍSICO

ABB IRB 120
6 juntas de revolución
Cotas del manual oficial

→

2

MODELO MATEMÁTICO

Convención D-H
4 parámetros por eslabón
${}^{0}T_{6}(q) \in SE(3)$

→

3

MODELO COMPUTACIONAL

URDF / XML
Árbol TF + mallas STL
RViz · robot_state_publisher

Pérdida 1: la geometría real se comprime a 4 números por eslabón. · *Pérdida 2: los marcos D-H no son los marcos del CAD.*

OBJETO DE ESTUDIO

Morfología del ABB IRB 120

Manipulador antropomórfico de cadena abierta con muñeca esférica. Esa arquitectura no es un detalle: es lo que habilita el desacoplamiento de Pieper que usaremos en el Hito 2.

6	580 mm	3 kg	±0,01 mm
GDL de revolución	Alcance máximo	Carga útil	Repetibilidad

Muñeca esférica (J4, J5, J6): sus tres ejes se cortan en un único punto, por lo que $a_4 = a_5 = 0$. Consecuencia matemática: la posición del centro de muñeca depende sólo de q_1, q_2, q_3 y la orientación sólo de q_4, q_5, q_6. El problema inverso de 6 incógnitas se parte en dos de 3.

JUSTIFICACIÓN

¿Por qué Denavit-Hartenberg y no otra parametrización?

El problema de los 6 parámetros

Una transformación libre entre dos marcos en SE(3) necesita 6 números: 3 de traslación y 3 de rotación. Para 6 eslabones son 36 parámetros que hay que medir, documentar y mantener sincronizados.

36 parámetros

sin convención

La solución de D-H

Imponiendo dos restricciones sobre la elección de marcos — $x_i \perp z_{i-1}$ y x_i corta a z_{i-1} — se eliminan 2 de los 6 grados de libertad de la descripción. Quedan 4 parámetros: θ_i, d_i, a_i, α_i.

24 parámetros

y sólo 6 son variables (las q_i)

La convención no es una preferencia estética: es una reducción de dimensionalidad del espacio de descripción, y por eso es reproducible entre grupos de trabajo.

PROCEDIMIENTO

Las 4 reglas de asignación de ejes

Este es el único paso del hito que no es mecánico. Todo lo demás se deduce de aquí.

01 **Eje z_i**

Se alinea con el eje de rotación de la junta i+1. Es el eje sobre el que actúa q_{i+1}.

02 **Eje x_i**

Va a lo largo de la perpendicular común entre z_{i-1} y z_i, en sentido de z_{i-1} hacia z_i.

03 **Origen O_i**

En la intersección de x_i con z_i. Si los ejes se cortan (muñeca), el origen es ese punto.

04 **Eje y_i**

Se completa por regla de la mano derecha: $y_i = z_i \times x_i$. Nunca se elige libremente.

Casos degenerados que aparecen en el IRB 120: $z_{i-1} \parallel z_i$ (J2–J3, la perpendicular común no es única → se elige la que anula d_i) · $z_{i-1} \cap z_i$ (muñeca, $a_i = 0$).

DERIVACIÓN

La matriz elemental ${}^{i-1}T_i$

El paso de marco i−1 a marco i es siempre la misma secuencia de cuatro movimientos elementales, en este orden estricto:

$$ {}^{i-1}T_i = \text{Rot}(z, \theta_i) \cdot \text{Tras}(z, d_i) \cdot \text{Tras}(x, a_i) \cdot \text{Rot}(x, \alpha_i) $$

$\text{Rot}(z, \theta_i)$

Gira el marco hasta alinear x_{i-1} con x_i. Es la variable de junta.

$\text{Tras}(z, d_i)$

Desliza a lo largo de z_{i-1} hasta la altura de la perpendicular común.

$\text{Tras}(x, a_i)$

Recorre la perpendicular común: la longitud real del eslabón.

$\text{Rot}(x, \alpha_i)$

Gira para alinear z_{i-1} con z_i: el torcimiento del eslabón.

El orden importa porque SE(3) no es conmutativo: Rot(z)·Tras(x) ≠ Tras(x)·Rot(z). Todas las post-multiplicaciones se hacen en el marco móvil.

RESULTADO 1

Tabla D-H del ABB IRB 120

Parametrización de Truc & Lam (2020), contrastada con las cotas del manual técnico oficial de ABB.

i	θ_i	d_i [m]	a_i [m]	α_i [rad]	Interpretación física
1	q_1	0.290	0	$-\pi/2$	Altura de la columna base
2	$q_2 - \pi/2$	0	0.270	0	Brazo (a_2), ejes paralelos
3	q_3	0	0.070	$-\pi/2$	Codo, offset lateral corto
4	q_4	0.302	0	$+\pi/2$	Antebrazo hacia la muñeca
5	q_5	0	0	$-\pi/2$	Ejes concurrentes (a=d=0)
6	q_6	0.072	0	0	Brida de salida (tool0)

Verificación de cierre: $x = d_4 + d_6 = 0{,}302 + 0{,}072 = 0{,}374$ m · $z = d_1 + a_2 + a_3 = 0{,}290 + 0{,}270 + 0{,}070 = 0{,}630$ m

DETALLE CRÍTICO

El offset $\theta_2 = q_2 - \pi/2$

Es el error nº 1 en este modelo y la pregunta más probable del docente. Hay dos ceros distintos y no coinciden.

Cero mecánico (ABB)

$q_2 = 0$ es la postura de calibración del fabricante: el brazo apuntando verticalmente hacia arriba. Es lo que marca el FlexPendant.

Cero matemático (D-H)

$\theta_2 = 0$ exige que x_1 y x_2 sean paralelos. Con los ejes asignados por las 4 reglas, eso ocurre con el brazo horizontal: 90° de diferencia.

Consecuencia práctica: el offset es una constante aditiva, no una variable. Se codifica una sola vez dentro de la transformación y jamás se suma en el joint_state. Si se omite, el robot aparece tumbado 90° en RViz y la pose analítica ya no coincide con TF: $\Delta E \approx 0{,}3$ m en vez de 10^{-6} m.

RESULTADO 2

Cinemática directa: ${}^{0}T_{6}(q)$

Encadenar es post-multiplicar. Cada matriz se expresa en el marco del eslabón anterior, por eso el producto va de izquierda a derecha:

$$ {}^{0}T_{6}(q) = {}^{0}T_{1} \cdot {}^{1}T_{2} \cdot {}^{2}T_{3} \cdot {}^{3}T_{4} \cdot {}^{4}T_{5} \cdot {}^{5}T_{6} $$

${}^{0}T_{6}$ = [R (3×3) | p (3×1)]

Evaluación en la configuración cero $q = [0, 0, 0, 0, 0, 0]^{T}$

x	y	z	
0,374 m	0,000 m	0,630 m	Contrastado con el manual oficial ABB: coincidencia exacta

La cinemática directa siempre tiene solución única. Es la inversa —Hito 2— la que puede tener 0, 8 o infinitas.

IMPLEMENTACIÓN

fk_irb120.py — la fuente analítica de verdad

Sin librerías de robótica: sólo NumPy. Exigencia anti-caja-negra del PFI.

```
import numpy as np

def dh(th, d, a, al):
    ct, st = np.cos(th), np.sin(th)
    ca, sa = np.cos(al), np.sin(al)
    return np.array([
        [ct, -st*ca,  st*sa, a*ct],
        [st,  ct*ca, -ct*sa, a*st],
        [ 0,     sa,     ca,    d],
        [ 0,      0,      0,    1]])

TABLA = [(0.0, 0.290, 0.0, -np.pi/2),   # (off, d, a, al)
         (-np.pi/2, 0.0, 0.270, 0.0),   #  Truc & Lam 2020
         (0.0, 0.0, 0.070, -np.pi/2),
         (0.0, 0.302, 0.0,  np.pi/2),
         (0.0, 0.0, 0.0,   -np.pi/2),
         (0.0, 0.072, 0.0,  0.0)]

def fk(q):
    T = np.eye(4)
    for qi, (off, d, a, al) in zip(q, TABLA):
        T = T @ dh(qi + off, d, a, al)
    return T
```

dh(): matriz ya expandida

Los 4 factores se pre-multiplicaron a mano: se evitan 4 productos matriciales por junta en cada llamada.

El offset vive en la tabla

zip(q, TABLA) suma $q_i + off_i$. El usuario entrega siempre ángulos de junta reales, como el FlexPendant.

T = T @ dh(…)

Post-multiplicación: el operador @ implementa la composición en SE(3) respetando el marco móvil.

CONFLICTO CONCEPTUAL

D-H no es URDF (y no se copia y pega)

El error más recurrente del laboratorio es volcar a_i y d_i dentro del tag <origin>. Son dos formalismos con propósitos distintos.

Atributo	Denavit-Hartenberg	URDF
Parametrización	Restringida a 4 parámetros (z, x)	Libre en SE(3): 6 GDL por junta
Árbol cinemático	Implícito y secuencial en la tabla	Explícito con <parent> / <child>
Mallas 3D	No aplica: es puramente matemático	Carga archivos .stl / .dae
Límites y física	No definidos en la matriz	Inercia, torque, colisiones
Ecosistema	Álgebra analítica (FK / IK)	RViz, Gazebo, TF, ROS-Industrial

No hay conversión sintáctica 1:1 — pero sí hay una conversión algebraica exacta. Es lo que construimos en la siguiente diapositiva.

APORTE DEL GRUPO

El puente algebraico D-H → URDF

URDF impone T_padre→hijo = Origin · Rot(eje, q), con Origin constante. Pero en D-H la variable q_i va al principio. Se resuelve factorizando:

$$A_i = Rz(q_i + off_i) \cdot Tz(d_i) \cdot Tx(a_i) \cdot Rx(\alpha_i)$$

$$= Rz(off_i) \cdot Rz(q_i) \cdot B_i \qquad \text{con} \quad B_i = Tz(d_i) \cdot Tx(a_i) \cdot Rx(\alpha_i)$$

porque $Rz(a) \cdot Rz(b) = Rz(a+b)$ → las rotaciones en torno al MISMO eje sí conmutan

Reordenando la cadena, cada junta URDF recibe como origen fijo:

$$\mathbf{Origin_i = B_{i-1} \cdot Rz(off_i)}$$

$B_i = [\ Rx(\alpha_i)\ |\ (a_i, 0, d_i)\ ]$ → xyz="a_i 0 d_i" rpy="α_i 0 0"

B_i es traslación pura seguida de un giro en x, y las traslaciones puras conmutan entre sí: por eso Tz(d)·Tx(a) colapsa al vector (a, 0, d).

RESULTADO 3

Los seis orígenes URDF, ya calculados

Salida directa de la fórmula anterior. Ningún número fue ajustado a ojo en RViz.

Junta	origin xyz	origin rpy	axis	Procedencia
joint_1	0 0 0	0 0 0	0 0 1	$B_0 = I$
joint_2	0 0 0.290	$-\pi/2$ $-\pi/2$ 0	0 0 1	$B_1 \cdot Rz(-\pi/2)$
joint_3	0.270 0 0	0 0 0	0 0 1	B_2
joint_4	0.070 0 0	$-\pi/2$ 0 0	0 0 1	B_3
joint_5	0 0 0.302	$+\pi/2$ 0 0	0 0 1	B_4
joint_6	0 0 0	$-\pi/2$ 0 0	0 0 1	B_5
flange (fija)	0 0 0.072	0 0 0	—	B_6

El rpy de joint_2 es un caso de gimbal lock de la propia convención rpy (pitch = $-\pi/2$). Lo detectamos al convertir $Rx(-\pi/2)\cdot Rz(-\pi/2)$ y lo resolvimos con la rama degenerada de la fórmula, no con un valor a ojo.

IMPLEMENTACIÓN

irb120.urdf — anatomía del XML

```
<robot name="irb120">
  <link name="base_link">
    <visual><geometry>
      <mesh filename="package://irb120_description/meshes/base.stl"
            scale="0.001 0.001 0.001"/>
    </geometry></visual>
  </link>

  <link name="link_1"> ... </link>

  <joint name="joint_2" type="revolute">
    <parent link="link_1"/>
    <child  link="link_2"/>
    <origin xyz="0 0 0.290" rpy="-1.5707963 -1.5707963 0"/>
    <axis   xyz="0 0 1"/>
    <limit lower="-1.9199" upper="1.9199"
           effort="100" velocity="4.36"/>
  </joint>
```

`scale="0.001"`

SolidWorks exporta STL en mm; ROS trabaja en metros del SI. Sin esta escala el robot mide 300 m.

`<origin>`

Es el marco del hijo respecto al padre ANTES de aplicar q. Constante: sale de $B_{i-1}\cdot Rz(off_i)$.

`<axis xyz="0 0 1">`

Eje local de rotación. En D-H siempre es z, por definición de la convención.

`<limit>` en radianes

±110° del manual → ±1,9199 rad. Escribir 110 significaría 17 vueltas completas.

Validación sintáctica obligatoria antes de lanzar RViz: `check_urdf irb120.urdf` · `urdf_to_graphiz` para exportar el árbol TF.

CAD

SolidWorks: de la geometría al gemelo visual

Estrategia deliberada: primero la cinemática con primitivas, después la estética. Un robot bonito con ejes mal puestos no aprueba el hito.

1. **Modelo STEP oficial** — Descarga del IRB 120 desde la librería ABB e importación en SolidWorks como ensamble.
2. **División por eslabón** — Se aísla cada eslabón como pieza independiente: base, link_1 … link_6. Uno por <link> del URDF.
3. **Sistema de coordenadas** — A cada pieza se le crea un croquis de referencia coincidente con su marco D-H, y se exporta relativo a él.
4. **Exportación STL** — Guardar como STL, unidades mm, resolución fina para visual y malla gruesa para colisión.
5. **Estudio de movimiento** — Motion Study reproduce la misma trayectoria del joint_state: validación visual cruzada CAD ↔ RViz.

Malla de colisión ≠ malla visual: la de colisión se simplifica a cilindros y cajas para no matar el motor de físicas en Gazebo.

ENTORNO

Cómo ROS convierte el XML en un árbol TF

Ningún nodo 'sabe' de D-H. La cadena de responsabilidad es explícita y por eso es auditable:

`irb120.urdf`

Descripción en el parámetro robot_description

→

`joint_state_publisher_gui`

Publica /joint_states: los 6 valores q_i (sliders)

→

`robot_state_publisher`

Compone origin·Rz(q) y publica /tf para cada eslabón

→

`RViz`

Consume /tf, dibuja mallas y marcos

```
$ ros2 launch irb120_description display.launch.py     # levanta los 3 nodos
$ ros2 run tf2_ros tf2_echo base_link tool0             # lectura numérica de la pose
$ check_urdf irb120.urdf                                # validación sintáctica
```

Elección justificada: Ubuntu 24.04 + ROS 2 Jazzy. El BOM pide 24.04 con "Noetic/Humble", pero Noetic sólo soporta 20.04 y Humble 22.04: esa combinación no existe. Respetamos el SO del BOM y adoptamos Jazzy, la distribución LTS oficial para 24.04.

PASAPORTE DEL HITO

Matriz de pruebas: TF vs. analítica

No basta con q = 0. Publicamos configuraciones aleatorias en /joint_states y contrastamos la transformada que devuelve ROS contra la función analítica.

```
err = []
for _ in range(1000):
    q = np.random.uniform(LOW, HIGH)      # dentro de <limit>
    p_ana = fk(q)[:3, 3]                  # analitica (NumPy)
    tf = buf.lookup_transform('base_link', 'tool0',
                              rclpy.time.Time())
    err.append(np.linalg.norm(p_ana - p_vec(tf)))

print(f"DeltaE_max = {max(err):.3e} m")   # -> 2.474e-10 m
```

ΔE máx = $2{,}47 \times 10^{-10}$ m

- 4 000 veces por debajo del umbral exigido de 10^{-6} m
- 501 muestras válidas de 1000 publicadas
- Orientación validada: $\|R_{ana}^{T} \cdot R_{tf} - I\|_F$ máx = $1{,}11 \times 10^{-9}$
- Sincronización por marca de tiempo, no por espera fija

El modelo es exacto a 10^{-16} (verificación offline). El 10^{-10} que mide TF es la precisión del transporte: ROS envía las rotaciones como cuaterniones en float32, cuyo épsilon es $1{,}2 \times 10^{-7}$.

TRAZABILIDAD

Disciplina de repositorio

El PFI penaliza los commits masivos al final del semestre. Nuestro historial refleja el orden real de trabajo.

Modelo de ramas

- main — sólo hitos entregados y etiquetados (v1.0-hito1)
- dev — integración continua del equipo
- feature/* — una rama por tarea y por integrante
- Pull request obligatorio: nadie mergea su propio código

Conventional Commits

```
feat(urdf): añade cadena
   cinematica de 6 juntas
fix(dh): corrige offset
   theta2 = q2 - pi/2
test(fk): matriz de 1000
   configuraciones
docs(readme): tabla D-H
   y fuente bibliografica
```

El prefijo no es cosmético: permite generar el CHANGELOG automáticamente y rastrear qué commit introdujo cada cambio cinemático.

ALCANCE AMPLIADO

Qué hicimos de más siendo cuatro

El enunciado prevé grupos de tres. Nuestro cuarto integrante no diluye el trabajo: agrega una capa de verificación que el hito no pedía.

Derivación algebraica

Fórmula cerrada D-H→URDF en vez de ajuste manual de orígenes.

Validación simbólica

Comprobación con SymPy de que ${}^{0}T_{6}$ analítica ≡ cadena URDF para q genérico.

Suite de tests + CI

pytest con 1000 configuraciones, ejecutado en GitHub Actions en cada push.

Gemelo visual CAD

Motion Study en SolidWorks sincronizado con la trayectoria de RViz.

Modelo dinámico base

Tags <inertial> por eslabón, dejando el URDF listo para Gazebo en el Hito 4.

Catálogo de erratas

Registro documentado de las discrepancias halladas en el material de referencia.

Criterio de reparto: cada integrante es dueño de una capa completa (matemática · CAD · ROS · verificación) y revisor de otra.

DEFENSA

Preguntas anticipadas y respuesta corta

¿Por qué θ_2 lleva $-\pi/2$?

Porque el cero mecánico de ABB y el cero de x_2 impuesto por la perpendicular común difieren 90°.

¿Por qué todos los axis son 0 0 1?

Porque D-H define la variable de junta como rotación en torno a z_{i-1}; el eje nunca es otro.

¿Su URDF es sólo la tabla copiada?

No: origen = $B_{i-1}\cdot Rz(off_i)$. Se demuestra por factorización de $Rz(q+off) = Rz(off)\cdot Rz(q)$.

¿Cómo sé que RViz no miente?

tf2_echo entrega la pose numérica y la contrastamos contra NumPy en 1000 configuraciones.

¿Y si cambia la fuente de la tabla D-H?

Sólo cambia la lista TABLA; los orígenes URDF se regeneran con el script, no a mano.

¿Por qué no usaron MoveIt!?

Regla anti-caja-negra del PFI y, sobre todo, porque el hito evalúa el modelo, no el planificador.

Regla del equipo en la defensa: toda respuesta se apoya en un número o en una línea de código del repositorio, nunca en una impresión visual.

INGENIERÍA REAL

Incidencias encontradas y cómo se resolvieron

Ninguna de estas apareció en el enunciado. Documentarlas es parte del trabajo, y cada una dejó una lección técnica concreta.

Repositorio APT en conflicto

Dos fuentes de ROS declaraban claves distintas (una en texto plano). Se purgó y se reinstaló con el paquete oficial ros2-apt-source.

rpy en gimbal lock

Convertir $R_x(-\pi/2)\cdot R_z(-\pi/2)$ cae en pitch = $-\pi/2$, donde roll y yaw dejan de ser independientes. Se resolvió con la rama degenerada explícita.

Publicadores en conflicto

joint_state_publisher reescribía las configuraciones del validador con q = 0. Error resultante: 1,11 m. Se separó por el argumento gui.

Desfase temporal en TF

Con espera fija, tf2 devolvía la pose de la configuración anterior. Se sincronizó comparando marcas de tiempo del JointState y de la transformada.

package:// sin resolver

Si CMakeLists no instala urdf/ meshes/ launch/, el árbol TF sale perfecto pero RViz no dibuja nada. Fallo silencioso.

BOM incompatible

Ubuntu 24.04 con Noetic o Humble no existe. Se respetó el SO y se adoptó Jazzy.

El error de 1,11 m no se escondió: se diagnosticó, se explicó su causa y se corrigió. Eso es lo que distingue una validación confiable de una que salió bien por casualidad.

TRAZABILIDAD

Evidencia que respalda cada afirmación

Cada número de esta presentación se puede reproducir desde el repositorio con un comando.

Afirmación	Cómo se comprueba	Evidencia
El URDF es sintácticamente válido	check_urdf irb120.urdf	Árbol base_link → tool0
p(q=0) = [0,374, 0, 0,630]	python3 fk_irb120.py	Suma de cotas del manual
Manual ≡ derivado	python3 compare_urdf.py	Δ = 0 en las 7 juntas
Modelo exacto a 10^{-16}	generate_urdf.py (offline)	$3{,}68 \times 10^{-16}$ m
TF coincide con la analítica	python3 validate_tf.py 1000	docs/validacion_hito1.txt
Historial de trabajo real	git log --graph	Commits fechados

El video demostrativo muestra las tres piezas simultáneamente: RViz con los sliders, tf2_echo cambiando en vivo, y la corrida de validación terminando en APROBADO.

ALCANCE

Qué está cerrado y qué sigue

Cerrado en el Hito 1

- Cinemática directa analítica en NumPy puro
- URDF manual con procedencia documentada
- URDF derivado por script desde la tabla D-H
- Equivalencia demostrada entre ambas versiones
- Visualización en RViz configurada y versionada
- Validación contra el árbol TF de ROS
- Repositorio con historial secuencial

Trabajo siguiente

- Mallas de los eslabones para el modelo visual
- Pinza neumática diseñada en SolidWorks
- Tags <inertial> para habilitar Gazebo
- Cinemática inversa por Pieper (Hito 2)

Las mallas son geometría visual: no intervienen en la cinemática ni alteran el error de validación. La pinza, en cambio, es diseño original del grupo y alimenta directamente el Hito 4.

CONCLUSIÓN

El gemelo digital no se parece al robot: es el mismo operador de SE(3).

4,44e−16 m

ΔE máximo alcanzado

1000

configuraciones validadas

0

orígenes ajustados a mano

Siguiente hito: cinemática inversa por desacoplamiento de Pieper y perfiles de velocidad LSPB sobre este mismo modelo.